

Alocação de Registradores em Síntese de Hardware Específico via Coloração de Vértices com Pesos

Programa de Pós Graduação em Ciência da Computação
Departamento de Ciência da Computação
Universidade Federal de Minas Gerais

Relatório Final de projeto da disciplina Projeto e Análise de Algoritmos

Pedro Ramos
pedroramos@dcc.ufmg.br

Resumo—O objetivo deste trabalho é o estudo algorítmico do problema de alocação de registradores na síntese de hardware para uma aplicação específica. Aplicações em C e C++, por terem modelos de memória estáticos, não possuem informação concreta sobre a largura em bits das variáveis. Síntese de hardware específico para aplicações nessas linguagens pode desperdiçar recursos. Técnicas de síntese de hardware avançadas utilizam larguras de variáveis baseadas em tipos, e portanto ainda assim não conseguem alcançar a economia de recursos ideal. Considerando-se que podemos projetar um hardware especificamente para uma aplicação, e que nosso banco de registradores pode conter um número infinito de registradores e que esses registradores podem ter tamanho variável, desejamos achar o menor número possível de registradores de tamanhos mínimos, evitando desperdício de recursos. Levamos em consideração o tamanho máximo de cada variável em bits e desejamos atribuir registradores às variáveis de forma que o tamanho de cada registrador seja mínimo. Observamos que este problema pode ser modelado em um problema de coloração de vértices com pesos, que é NP-Difícil. Apresentaremos uma heurística como baseline e propomos uma nova metaheurística híbrida baseada em algoritmos evolucionários. Realizamos a análise de complexidade da heurística proposta e testamos ambas as heurísticas em dois benchmarks diferentes: um contendo grafos genericamente gerados, e outro contendo grafos de intervalos das variáveis no SPEC CPU 2006.

I. INTRODUÇÃO

Em otimização de compiladores, o problema de alocação de registradores consiste em um processo de atribuição de um largo número de variáveis a um restrito número de registradores em CPU [1]. Em programas no formato SSA (*Static Single Assignment*) o problema de alocação de registradores pode ser modelado como um problema de coloração de vértices. [1], [2], [3].

A modelagem do problema em um grafo é feita a partir da geração do grafo de intervalos (ou grafo de interferências) das variáveis de um programa, que é gerado em uma análise de fluxo. No grafo de intervalos das variáveis, cada vértice representa uma variável do programa. A geração do grafo de intervalos depende intimamente de uma análise de fluxo bem conhecida em otimização de código chamada Análise de Variáveis Vivas [4]. Esta análise fornece o intervalo de vida das variáveis no programa; quando uma variável não está mais viva, consequentemente ela não será mais utilizada e pode ser "cuspada" (do inglês *spilling*) para fora da memória. Dessa forma, quando duas variáveis estão vivas ao mesmo tempo no programa, é ideal que elas estejam disponíveis

em registradores. Portanto, no grafo de intervalos, quando duas variáveis estão vivas ao mesmo tempo, há uma aresta conectando os dois vértices que as representam.

A figura 1 ilustra um grafo de intervalos. Sejam A, B, C, D, E, F e G variáveis do programa, e seus respectivos intervalos de vida ilustrados na imagem. Quando duas variáveis estão vivas ao mesmo tempo, há uma aresta entre elas no grafo.

A partir do grafo de intervalos é fácil perceber que o problema de alocar o menor número de registradores possível para que as variáveis estejam disponíveis o tempo todo em registradores enquanto estiverem vivas no programa é equivalente a achar uma coloração ótima no grafo de intervalos. Dois vértices adjacentes não podem estar no mesmo registrador ao mesmo tempo, pois representam variáveis que estão vivas ao mesmo tempo no programa, e portanto não podem ter a mesma cor. Cada cor representa um registrador.

No problema em questão, há um limite de recursos, isto é, um número de registradores disponíveis limitado. Portanto não basta achar a melhor coloração; é preciso achar uma coloração com K cores, que representa o número de registradores disponíveis, que minimize a quantidade de *spillings* no programa. Isto é, quando duas variáveis foram atribuídas ao mesmo registrador, deverá haver operações de *load* e de *store* durante a execução do programa, que carregam aquela variável na memória e no registrador à medida que for necessário. Portanto quando se há um número limitado de cores/registradores, e não existir uma coloração ótima com este número de cores, é preciso achar uma coloração que minimize o número de operações de *load* e *store*, que são caras para o processador [2].

O problema de alocação de registradores é NP-Difícil, pois o problema de coloração de vértices é NP-Difícil [5]. Alocação de Registradores já foi um problema muito atacado em Ciência da Computação, e atualmente a heurística estado da arte é a do algoritmo *Linear Scan* [6].

A. Alocação de Registradores e Análise de Largura de Variáveis

A proposta deste trabalho pretende atacar um problema diferente, que pode ser considerado uma generalização do problema de coloração de vértices, porém com pesos nos vértices.

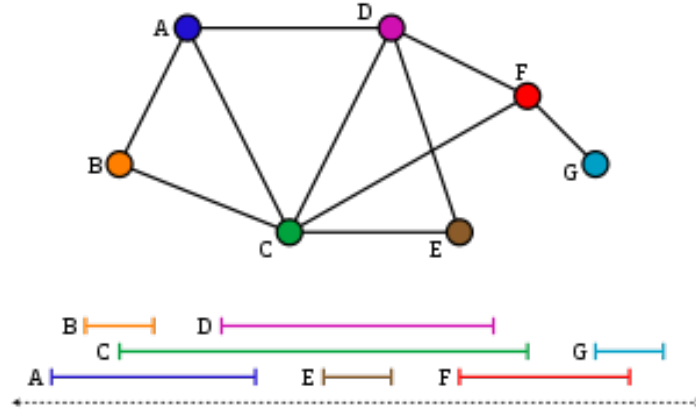


Figura 1: Exemplo de Grafo de Intervalos

Em hardware sintetizados para aplicações específicas, o problema de alocação de registradores ganha uma nova dimensão [7]. Supondo-se que seja possível saber o valor máximo que uma variável ocupa em *bits*, e que há um número infinito de registradores disponíveis, podemos formalizar o problema de alocação de registradores de uma outra forma. Seja o valor máximo de cada variável o peso w no vértice V_i que a representa. Desejamos achar uma coloração ótima de modo que o peso máximo de cada cor seja mínimo, ou seja, queremos a menor quantidade de registradores necessária para aquele programa, e queremos os menores registradores possíveis.

Este problema pode ser reduzido ao problema de coloração de vértices com pesos, e foi descrito inicialmente em [8] e abordado em diversos trabalhos como [9], [10], [11], [12], [13].

É possível determinar o valor máximo de uma variável utilizando a análise de fluxo chamada *Análise de Largura de Variáveis* (Range Analysis, em inglês) [14]. Esta análise nos dá o valor mínimo e máximo que uma variável pode assumir durante toda a execução do problema. Casado à alocação de registradores, podemos modelar o problema como uma generalização do problema de coloração de vértices, porém com vértices que tenham peso, na qual o objetivo é não só minimizar o número de cores (registradores) mas também o peso máximo em cada cor (tamanho máximo de cada registrador).

II. MODELAGEM EM GRAFOS

Para um programa cujas variáveis tenham largura de bits uniforme, o problema de minimizar o número de registradores pode ser reduzido para encontrar o número cromático em um grafo de intervalos, que pode ser resolvido por um algoritmo polinomial [1], [15], [6]. Considerando registradores com diferentes larguras em bits, o novo problema é minimizar a largura em bits total dos registradores.

Para modelar o problema em grafos, precisamos estabelecer alguns conceitos. Seja $s(a)$ o ponto em que uma variável a é definida, e $t(a)$ o ponto em que a variável a é utilizada pela última vez no programa. Dizemos que a está viva entre $s(a)$ e $t(a)$, contanto que não haja nenhuma redefinição de

a neste caminho¹ [16]. Sendo assim, um **grafo de intervalos com pesos nos vértices** $G(V, E)$ pode ser derivado, em que cada vértice em V corresponde a uma variável no programa e uma aresta $(a_1, a_2) \in E$ se e somente se as vidas das variáveis a_1 e a_2 se sobrepõem. Ou seja, se duas variáveis estão vivas em algum momento no programa, então há uma aresta E entre os dois vértices que a representam no grafo. Portanto fica claro que duas variáveis podem ser associadas ao mesmo registrador **somente se elas não estão vivas ao mesmo tempo no programa**. Dessa forma, uma coloração P para este grafo G corresponde à uma associação de variáveis à registradores que representa uma solução para o problema. Todas as variáveis de uma mesma cor podem ser associadas ao mesmo registrador.

Seja também $w(v)$ o *peso do vértice* v , em que $w(v) = b(a)$, em que $b(a)$ é a largura em bits da variável a correspondente de v . Definimos como o peso de um *conjunto de vértices* V como sendo $W(V) = \max\{w(v) | v \in V\}$, e o *peso do grafo* $G(V, E)$ como $W(G) = W(V)$. Suponhamos que uma coloração $P = \{c_1, c_2, \dots, c_k\}$ de $G(V, E)$ particione V em subconjuntos de vértices $V_1, V_2, \dots, V_k$ e o peso da cor c_i para $1 \leq i \leq k$ é definido como $W(V_i)$, então o peso da coloração P é definido como

$$W(P) = \sum W(V_i)$$

Definição do Problema. Dessa forma, uma alocação de registradores sensível à largura em bits das variáveis é equivalente à coloração de um grafo de intervalos com pesos nos vértices. Então, **dado** um grafo de intervalos $G(V, E)$, o **objetivo** é descobrir uma coloração P de G , tal que o peso da coloração $W(P)$ seja mínimo.

A figura 2 demonstra um cenário em que há 6 variáveis e suas larguras em bits e suas linhas de vida. Em (b) vemos a construção do grafo de intervalos a partir da informação das linhas de vida casada à informação da largura das variáveis.

Embora o problema de coloração de vértices em grafos de intervalos possa ser resolvido em tempo polinomial [1], [6],

¹Como lidamos com programas na forma SSA, há apenas uma definição de cada variável, de forma que a análise de vida das variáveis é mais simples [4]

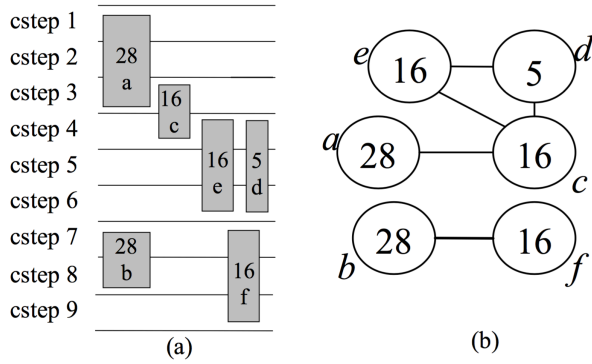


Figura 2: Exemplo de Grafo de Intervalos com Largura de Variáveis

[15], o problema de coloração de vértices com pesos em grafos de intervalos é NP-Difícil [11]. Como o mesmo problema de otimização na forma canônica é NP-Difícil [8], trataremos de heurísticas comparativas à complexidade do problema na forma canônica.

III. HEURÍSTICAS E TESTES PRELIMINARES

Nesta seção descrevemos brevemente as heurísticas mais abordadas na literatura para o problema de coloração de vértices com pesos. Apresentamos também uma heurística como nosso *baseline* e realizamos testes preliminares em grafos genéricos para comprovar seu comportamento assintótico.

A. Heurísticas na Literatura

Existem diversas heurísticas construtivas para solucionar o problema de coloração de vértices [12]. A maioria envolve a construção de uma coloração construtivamente e de forma gulosa, ou seja, baseada em busca local, à medida que percorre-se a lista de vértices. Suas variações geralmente envolvem a ordem em que se avalia os vértices. Essas heurísticas são conhecidas como heurísticas do tipo *first fit*. Colore-se os vértices um por um, utilizando a cor mais utilizada disponível para colorir o próximo. Suas variantes são:

- 1) LDO (Least Degree Ordering): Ordena os vértices por ordem decrescente de grau. A ideia é que vértices de maior grau possam ser coloridos mais facilmente.
- 2) SDO (Saturation Degree Ordering): É uma variante do LDO, em que os vértices são ordenados em ordem decrescente de "grau de saturação", definido como o número de cores distintas na vizinhança do vértice.
- 3) IDO (Incidence Degree Ordering): É uma variante do SDO em que o grau de um vértice é definido como o número de vértices já coloridos em sua vizinhança.

Construções sequenciais são métodos extremamente rápidos, pois são lineares no número de vértices [17], porém pouco eficientes em algumas instâncias, pois podem conferir um número cromático distante do limite mínimo calculado.

Muitas metaheurísticas utilizam pesquisa tabu (*Tabu Search*) [18], [19], [20], [21], [22]. Pesquisa tabu é um procedimento adaptativo auxiliar, que guia um algoritmo de busca local na exploração contínua dentro de um espaço de busca. A partir de uma solução inicial, tenta avançar para uma outra solução (melhor que a anterior) na sua vizinhança até que se satisfaça um determinado critério de parada.

Há também registro de outras metaheurísticas que utilizam arrefecimento simulado [23], [24], busca local iterada [25], [26] busca em vizinhança variável [27], [28] algoritmo genético [29], [30], [31], [32], [33], [34], [35] e outras abordagens híbridas [22], [30], [36].

Para o problema de coloração de vértices com pesos, [37] propõe uma heurística híbrida de descida de vizinhança variável com *backtracking* que é dividida em três partes. Na primeira, o grafo é pré-processado para reduzir o seu tamanho e adequar o conjunto de vértices a uma heurística construtiva. Na segunda parte, uma solução inicial é construída pela heurística construtiva e por final, na terceira etapa há a busca em vizinhança variável com *backtracking* que processa a solução inicial reduzindo o custo da solução final.

Duas alternativas são propostas em [12], todas baseadas em Programação Linear Inteira. Uma delas é utilizada para derivar um limite inferior firme para o número cromático e a outra é utilizada para derivar uma heurística de duas fases cujo objetivo é melhorar a qualidade das soluções encontradas.

B. Baseline: Coloração de Grafos de Intervalos com Pesos nos Vértices

Em relação ao algoritmo exato, um algoritmo de *Branch Bound* exato é proposto em [38], capaz de resolver grafos aleatórios de até 90 vértices. Porém não nos é útil, uma vez que o número de variáveis em uma aplicação pode ser muito superior a este número. Portanto nosso *baseline* não pode ser o algoritmo exato.

Escolhemos uma heurística descrita em [39], utilizada exatamente para o mesmo propósito - síntese de hardware específico - como nosso *baseline*. Ela será chamada de Heurística IGC a partir deste ponto. A descreveremos a seguir.

Utilizaremos um pequeno exemplo para ilustrar como a heurística IGC calcula o limite inferior do peso das colorações para um grafo de intervalos ponderado nos vértices. A figura ??(a) mostra as linhas de vida de 5 variáveis, com suas larguras de bit anotadas, e a figura 3(b) é o grafo de intervalos derivado $G(V, E)$. Primeiramente dividimos G em subgrafos de acordo com o peso dos vértices. Todos os vértices em cada subgrafo tem o mesmo peso. É óbvio que esses subgrafos e suas uniões ainda mantém a característica de ser um grafo cordal. Então ordenamos estes subgrafos em ordem decrescente de pesos. Por exemplo, na figura 3, os subgrafos ordenados serão $G_1 = \{a, b\}$, $G_2 = \{c, e, f\}$ e $G_3 = \{d\}$. Sabemos, ao observar G_1 , que o número de cores com peso igual a 28 deve ser no mínimo $\chi(G_1) = 1$ para qualquer coloração de G . Então para $G_1 \cup G_2$, o número de cores com peso no mínimo 16 deve ser pelo menos $\chi(G_1 \cup G_2)$. Combinando esses dois fatos, concluímos que o número de cores com peso 16 é pelo menos $\chi(G_1 \cup G_2) - \chi(G_1) = 1$. Similarmente para $G_1 \cup G_2 \cup G_3$, a cor com peso 5 deve ser no mínimo $\chi(G_1 \cup G_2 \cup G_3) -$

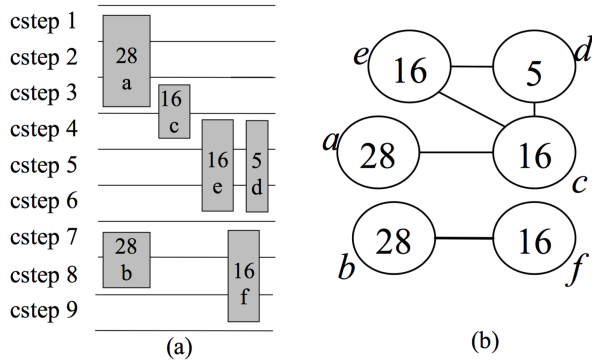


Figura 3: Linhas de vida das variáveis (a) e o grafo de intervalos (b) derivado das mesmas.

$\chi(G_1 \cup G_2) = 1$. Baseado nesses resultados, concluímos que o limite inferior do peso de qualquer coloração para esse grafo de intervalos é $28 * 1 + 16 * 1 + 5 * 1 = 49$.

A heurística IGC utiliza as ideias apresentadas anteriormente para calcular o limite inferior do peso de qualquer coloração para solucionar o problema de coloração em um grafo de intervalos ponderado nos vértices. Novamente utilizaremos a figura 3(b) para ilustrar a solução da heurística IGC. Após o particionamento e ordenação de G em subgrafos $G_1 = \{a, b\}$, $G_2 = \{c, e, f\}$ e $G_3 = \{d\}$, primeiramente tentamos mover os vértices de G_i , para $2 \leq i \leq 3$, para G_1 , a fim de decrescer o número cromático de cada G_i sem acrescer o número cromático de G_1 , representado por $\chi(G_1)$. Uma vez que G_2 tem o maior peso entre os subgrafos, reduzir $\chi(G_2)$ é preferível. Observamos que o número cromático χ é igual ao tamanho do clique máximo para um grafo de intervalos, então sempre tentamos mover os vértices dos cliques maximais de G_i para G_1 . Dessa forma, $\chi(G_i)$ pode ser diretamente decrescido. Para G_2 , o único clique maximal é $\{c, e\}$. Mover c de G_2 para G_1 decresce $\chi(G_2)$ em 1, mas aumenta $\chi(G_1)$ em 1 também. Entretanto, mover e seguramente diminui $\chi(G_2)$ sem incrementar $\chi(G_1)$. Portanto, e é escolhido para ser adicionado em G_1 . Uma vez que nenhum vértice no novo $G'_2, \{c, f\}$, pode ser seguramente movido para dentro de G_1 , continuamos e passamos a processar G_3 . Adicionando d em G_1 aumenta $\chi(G_1)$, então não podemos fazer nenhum movimento em G_3 . Agora, o processamento em G_1 está terminado. Agora amos processar G_i , para $2 \leq i \leq 3$ da mesma forma.

A partição final de G é $G_1 = \{a, b, e\}$, $G_2 = \{c, f\}$ e $G_3 = \{d\}$, e a coloração será $P = \{\{a, b, e\}, \{c, f\}, \{d\}\}$ com peso 49, que é igual ao limite inferior calculado anteriormente.

A figura 4 representa o pseudo-código da heurística IGC. O algoritmo é descrito a seguir.

Durante o processamento do subgrafo G_i , tentamos mover vértices de G_j , para $i + 1 \leq j \leq m$, para G_i . O processo de mover consiste em duas fases, ambas processam os subgrafos em ordem decrescente de seus pesos. A primeira fase seleciona um vértice de cada clique maximal de G_j para compor um conjunto de vértices S e mesclá-lo com G_i , de tal forma

```

Procedure Weighted_Interval_Graph_Coloring
Input:
    Weighted interval graph  $G$ 
Output:
    Coloring scheme of  $G$ 

Partition and sort  $G$  into  $\{G_1, G_2, \dots, G_m\}$ , with the decreasing order of the weights
for each  $G_i$ ,  $1 \leq i \leq m-1$ 
    Compute  $\chi(G_i)$ 
    for each  $G_j$ ,  $i+1 \leq j \leq m$ 
        while  $\exists S \subseteq G_j$ , such that  $\chi(G_j - S) = \chi(G_j) - 1$  and  $\chi(G_i \cup S) = \chi(G_i)$ 
             $G_i \leftarrow G_i \cup S$ 
             $G_j \leftarrow G_j - S$ 
        for each  $G_j$ ,  $i+1 \leq j \leq m$ 
            while  $\exists v \in G_j$ , such that  $\chi(G_i \cup \{v\}) = \chi(G_i)$ 
                 $G_i \leftarrow G_i \cup \{v\}$ 
                 $G_j \leftarrow G_j - \{v\}$ 
Coloring each new subgraph separately

```

Figura 4: Algoritmo IGC.

que o número cromático de G_i não aumente. Uma vez que a cardinalidade do clique maximal é igual ao número cromático em grafos de intervalos, decrescer a cardinalidade de todos os cliques máximos em G_j irá diretamente reduzir o número cromático. Então, tirar o conjunto S de G_j decresce $\chi(G_j)$ em 1.

Quando nenhum vértice de G_j , para $i + 1 \leq j \leq m$, pode ser movido para G_i na primeira fase, a segunda fase é executada. Agora selecionamos aleatoriamente um vértice de G_j para mover para dentro de G_i sem acrescer $\chi(G_i)$. O propósito dessa fase é maximizar a utilização de cores com peso grande, e então providenciar mais chances de redução do número de cores com pesos pequenos.

C. Heurística proposta

Propomos uma metaheurística baseada em algoritmos evolucionários para encontrar uma coloração para um grafo cordal com pesos nos vértices. Nossa heurística segue um modelo de algoritmo evolucionário em que cada indivíduo é uma coloração própria para o grafo. Sabemos que encontrar o número cromático em um grafo de intervalos é possível em tempo polinomial [6]. Nossa heurística, chamada de Coloração Genética (a partir daqui referida-se como CG) funciona da seguinte maneira.

- 1) **Descrição do Indivíduo:** cada indivíduo da heurística é uma coloração P que atribui uma cor c_i a cada vértice v de tal forma que para toda aresta $E = (u, v)$, $c(v) \neq c(u)$. Cada coloração P tem também um peso W que é o peso total de cada cor, em que $W(P) = \sum W(V_i)$. Dessa forma cada indivíduo é uma coloração própria para o Grafo de Intervalos G .
- 2) **Geração da População inicial:** Para gerar a população inicial, primeiro computamos $\chi(G)$, que é o número cromático. Computar o número cromático em grafos cordais é feito em tempo polinomial. A partir do número cromático, geramos colorações P aleatórias com o número cromático $\chi(G)$.
- 3) A cada geração, há uma chance de que dois indivíduos cruzem e gerem filhos, e há uma chance de ocorrer mutação. Ambas as operações serão descritas a seguir. O cruzamento é feito por torneio de tamanho N . Os dois melhores indivíduos entre uma amostra aleatória

Nº Vértices	Nº Cromático	W(P)	% Ganho em Recursos	Tempo (s)
100	37	394	66.72%	0,004
200	165	1324	74.92%	0,011
400	111	908	74.44%	3,31
800	321	3801	63.00%	11,27
1600	434	4435	68.07%	36,21
3200	915	9899	66.19%	102,93
6400	1094	14089	59.75%	228,16
10000	3173	24674	75.70%	310,74

Tabela I: Resultado da Heurística IGC em grafos cordais genéricos. $W(P)$ representa o peso da coloração P ; O Ganho em Recursos indica o quanto em termos de bits desperdiçados foi poupado levando-se em consideração a largura das variáveis e não somente os registradores a elas associados.

de N indivíduos quaisquer da população são escolhidos para o cruzamento.

- 4) Ao final de cada geração, selecionam-se apenas os 70% melhores indivíduos e os outros 30% são gerados aleatoriamente como no passo 2. O algoritmo termina ao final de M gerações.

A *fitness* do indivíduo, isto é, a avaliação da qualidade do indivíduo, é simplesmente o peso $W(P)$. Quanto menor, melhor. As colorações de menor peso são as melhores.

O *cruzamento* é feito da seguinte forma. Dados dois indivíduos que são colorações, troca-se dois subconjuntos de vértices V_1 e V_2 (tamanho aleatório) de cada indivíduo P_1 e P_2 respectivamente. Se a restrição de cores adjacentes diferentes for quebrada após a troca, muda-se a cor de forma gulosa até que as restrições sejam respeitadas. Desse cruzamento surgem 2 filhos: um que herdou o subconjunto de P_1 e outro que herdou o subconjunto de P_2 . Compara-se os 4 indivíduos e seleciona-se os dois melhores da família.

A *mutação* é mais simples: para cada indivíduo, há uma chance PM de sofrer mutação. A mutação é simplesmente a troca de cores entre dois conjuntos de vértices quaisquer. A mutação pode levar a uma coloração pior.

Para evitar a convergência para um mínimo local, que é um problema comum em metaheurísticas genéticas, aumentamos a taxa de mutação e geramos 30% de indivíduos novos a cada geração.

IV. EXPERIMENTOS

A. Testes preliminares: Baseline IGC

Implementamos a heurística baseline (IGC) em grafos cordais genéricos de tamanho $|V| = 100$ a $|V| = 10.000$. Os pesos aleatórios variam de 1 a 32 bits. O comportamento temporal medido da heurística nos leva a confirmar a complexidade assintótica da heurística de $O(n^3)$, como observado na tabela I.

O ganho em recursos é representado pela diferença ao utilizar a análise de largura e otimizar também o peso máximo de cada cor c_i em relação a caso levássemos em consideração somente o número cromático, isto é, cada registrador (cor c_i) com o tamanho máximo possível (32 bits).

B. Testes Preliminares: Heurística CG

sec sec : testsecg

A metaheurística foi executada com os seguintes parâmetros na tabela II.

Nº Gerações	PM	PC	Torneio	Nº Indivíduos
100	5%	95%	10	100

Tabela II: Parâmetros utilizados na execução do algoritmo Coloração Genética. PM significa Probabilidade de Mutação a cada geração, PC significa Probabilidade de Cruzamento entre os dois melhores do Torneio, Torneio é o tamanho do torneio em número de indivíduos.

A operação mais cara da heurística CG é o cruzamento, que envolve a troca de subgrafos. Essa operação define a complexidade do nosso algoritmo. A mutação envolve uma troca simples de cor, que no pior caso é linear, mas na prática observamos que é $O(1)$. Considerando-se que um cruzamento é $O(n)$, e que são feitas 50 tentativas de cruzamento com probabilidade de 95%, e que o mesmo processo acontece em cada uma das 100 gerações, definimos a complexidade média do nosso algoritmo como sendo $\theta(475 * n)$.

Os resultados da nossa heurística são verificados na tabela III. Podemos fazer várias observações. Percebemos que a heurística genética demora muito e tem uma escalabilidade alta, pois foi inviável executá-la para os grafos genéricos acima de 3200 nós. Outra observação clara é a ineficiência da heurística coloração genética quando os grafos são muito grandes. Uma suposição para explicar esse ocorrido é que as operação de cruzamento e mutação, responsáveis por gerar diversidade na população e evitar a convergência do algoritmo para um mínimo local, tem pouco impacto sobre um indivíduo quando ele possui muitos nós. Os tempos de execução nos permite observar a lentidão e escalabilidade da heurística proposta.

O gráfico da figura 5 mostra a escalabilidade e inviabilidade da heurística.

C. Dataset real

Para realizar uma avaliação completa da nossa heurística, o plano inicial era utilizar um *benchmark* real, que representasse com fidelidade o propósito da aplicação: alocação de registradores. Utilizaríamos o *SpecCPU2006* [40]. Para gerar o grafo de intervalos $G(V, E)$ que é entrada para a heurística, primeiro pré-processamos os programas do *benchmark* e extraímos os conjuntos das variáveis vivas e suas larguras em

Nº Vértices	Nº Cromático	W(P)	% Ganho em Recursos	Tempo (min)
100	37	512	56.76%	2
200	165	3167	40.02%	7
400	111	2778	21.79%	23
800	321	8769	14.63%	61
1600	434	11620	16.33%	163
3200	915	25934	11.43%	329
6400	1094	-	-	-
10000	3173	-	-	-

Tabela III: Resultados da heurística CG nos mesmos grafos cordais genéricos.

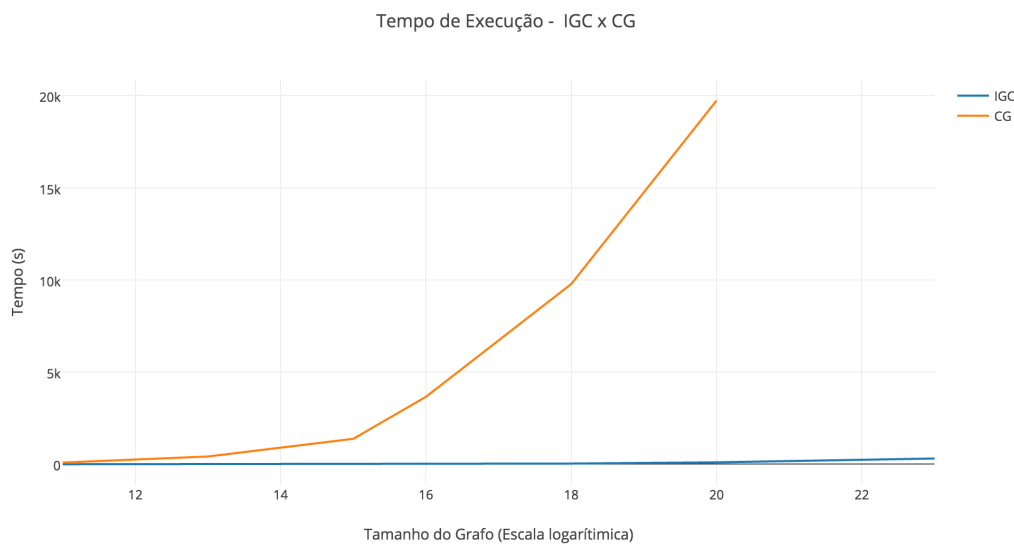


Figura 5: Comparação entre o tempo de execução das duas heurísticas sobre grafos cordais genéricos.

Programa	Nº de Vértices
milc	2325
namd	12587
dealII	78682
soplex	8062
lbm	244
perlbench	26948
bzip2	1684
gcc	62477
mcf	374
gobmk	13969
hmmer	5502
sjeng	2296
libquantum	925
h264ref	11586
omnetpp	22447
astar	864

Tabela IV: Número de vértices no Grafo de intervalos gerados a partir da análise da IR dos programas em SpecCPU2006

bits. Dessa forma cada variável pode ter uma largura de 1 a 32 bits (considerando-se uma arquitetura de 32 bits). Esse processo é feito no LLVM v.3.7 ², uma infra-estrutura que permite a análise e transformação/otimização de programas em uma representação intermediária. Tanto a análise de variáveis vivas quanto a análise de largura de variáveis são feitas sobre o programa na representação intermediária do LLVM. Nosso grafo de intervalos $G(V, E)$ tem então vértices cujo peso é um número inteiro que varia de 1 a 32, e as arestas são criadas conforme descrito na modelagem (ver seção II).

Portanto desenvolvemos todo o nosso código como um passe para o LLVM. A análise de variáveis vivas, que é uma análise necessária para se extrair os conjuntos de variáveis vivas ao mesmo tempo que gerarão os grafos de intervalos, foi feita e executada sobre o *SPEC CPU2006*, e os dados dos grafos de intervalos gerados a partir dessa análise podem ser observados na tabela IV.

Entretanto, devido a escalabilidade da nossa heurística, não

fomos capazes de executá-la sobre o SPEC como planejado. Ao invés disso, executamos ela somente sobre os grafos cordais genéricos como descrito na seção anterior. Pedimos desculpas pelo mal planejamento do projeto, afinal esse era o plano de experimentos na proposta inicial.

Contudo, executamos nossa heurística (CG) e a heurística IGC sobre um dos programas do SPEC: o *gcc*. O *gcc*, que é o maior programa da lista, gera um grafo de intervalos com 62.477 vértices que representam variáveis. Devido à modularização do código, este é um grafo altamente desconectado. Isso permite que nossa heurística execute com mais facilidade as operações de cruzamento (que envolvem a troca de dois subconjuntos de vértices). A execução da heurística IGC demorou 40 minutos, e a nossa demorou mais de 12 horas. Os resultados não foram favoráveis: a heurística IGC apresenta um resultado extremamente superior, fornecendo um ganho de 45% de recursos contra apenas 23% sobre a heurística CG.

V. CONCLUSÃO

Este trabalho foi importante para o estudo algorítmico de problemas intratáveis. Entretanto, houve certa dificuldade em implementar as heurísticas para aplicação prática, visto que os datasets reais são muito grandes. Alocação de registradores é um passo importante durante a projeção de um compilador, e uma heurística muito lenta pode inviabilizar o uso da mesma. Entendemos também que a heurística CG poderia ter sido melhorada e polida, se houvesse mais tempo para alterar os parâmetros e testar novas configurações. O desafio de se propor um alocador de registradores que leve em consideração a largura em bits das variáveis, a fim de se construir um hardware específico para uma aplicação com o intuito de poupar recursos (isto é, construir apenas os registradores necessários) permanece um desafio grande em arquitetura e projeto de hardware, bem como em compiladores.

²<http://llvm.org/releases/3.7.0/docs/GettingStarted.html>

REFERÊNCIAS

- [1] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein, "Register allocation via coloring," *Computer languages*, vol. 6, no. 1, pp. 47–57, 1981.
- [2] G. Chaitin, "Register allocation and spilling via colouring," *SIGPLAN Notices*, vol. 17, no. 6, 1982.
- [3] S. Hack, D. Grund, and G. Goos, "Register allocation for programs in ssa-form," in *International Conference on Compiler Construction*. Springer, 2006, pp. 247–262.
- [4] F. Brandner, B. Boissinot, A. Darte, B. D. De Dinechin, and F. Rastello, "Computing liveness sets for ssa-form programs," Ph.D. dissertation, INRIA, 2011.
- [5] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of computer computations*. Springer, 1972, pp. 85–103.
- [6] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 21, no. 5, pp. 895–913, 1999.
- [7] G. Hempel, J. Hoyer, T. Pionteck, and C. Hochberger, "Register allocation for high-level synthesis of hardware accelerators targeting fpgas," in *Reconfigurable and Communication-Centric Systems-on-Chip (ReCoSoC), 2013 8th International Workshop on*. IEEE, 2013, pp. 1–6.
- [8] D. Guan and Z. Xuding, "A coloring problem for weighted graphs," *Information Processing Letters*, vol. 61, no. 2, pp. 77–81, 1997.
- [9] D. De Werra, M. Demange, B. Escoffier, J. Monnot, and V. T. Paschos, "Weighted coloring on planar, bipartite and split graphs: complexity and improved approximation," in *International Symposium on Algorithms and Computation*. Springer, 2004, pp. 896–907.
- [10] M. Čangalović and J. A. Schreuder, "Exact colouring algorithm for weighted graphs applied to timetabling problems with lectures of different lengths," *European Journal of Operational Research*, vol. 51, no. 2, pp. 248–258, 1991.
- [11] B. Escoffier, J. Monnot, and V. T. Paschos, "Weighted coloring: further complexity and approximability results," *Information Processing Letters*, vol. 97, no. 3, pp. 98–103, 2006.
- [12] E. Malaguti, M. Monaci, and P. Toth, "Models and heuristic algorithms for a weighted vertex coloring problem," *Journal of Heuristics*, vol. 15, no. 5, pp. 503–526, 2009.
- [13] D. Cornaz, F. Furini, and E. Malaguti, "Solving vertex coloring problems as maximum weight stable set problems."
- [14] W. H. Harrison, "Compiler analysis of the value ranges for variables," *IEEE Transactions on software engineering*, no. 3, pp. 243–250, 1977.
- [15] G. D. Micheli, *Synthesis and optimization of digital circuits*. McGraw-Hill Higher Education, 1994.
- [16] F. E. Allen, "Control flow analysis," in *ACM Sigplan Notices*, vol. 5, no. 7. ACM, 1970, pp. 1–19.
- [17] J. Fijuljanin, "Two genetic algorithms for the bandwidth multicoloring problem," *Yugoslav Journal of Operations Research ISSN: 0354-0243 EISSN: 2334-6043*, vol. 22, no. 2, 2012.
- [18] A. E. Eraghi, J. A. Torkestani, and M. R. Meybodi, "„solving the bandwidth multicoloring problem: a cellular learning automata approach “,” in *Proceedings Of the 2009 International Conference On Machine Learning And Computing*, 2009.
- [19] I. Blöchliger and N. Zufferey, "A graph coloring heuristic using partial solutions and a reactive tabu scheme," *Computers & Operations Research*, vol. 35, no. 3, pp. 960–975, 2008.
- [20] R. Dorne and J.-K. Hao, "Tabu search for graph coloring, t-colorings and set t-colorings," in *Meta-heuristics*. Springer, 1999, pp. 77–92.
- [21] J. Ferland and C. Fleurent, "Object-oriented implementation of heuristic search methods for graph coloring, maximum clique and satisfiability," Univ. of Michigan, Ann Arbor, MI (United States), Tech. Rep., 1994.
- [22] A. Hertz and D. de Werra, "Using tabu search techniques for graph coloring," *Computing*, vol. 39, no. 4, pp. 345–351, 1987.
- [23] M. Chams, A. Hertz, and D. De Werra, "Some experiments with simulated annealing for coloring graphs," *European Journal of Operational Research*, vol. 32, no. 2, pp. 260–266, 1987.
- [24] D. S. Johnson, C. R. Aragon, L. A. McGeoch, and C. Schevon, "Optimization by simulated annealing: an experimental evaluation; part ii, graph coloring and number partitioning," *Operations research*, vol. 39, no. 3, pp. 378–406, 1991.
- [25] M. Chiarandini, T. Stützle *et al.*, "An application of iterated local search to graph coloring problem," in *Proceedings of the Computational Symposium on Graph Coloring and its Generalizations*, 2002, pp. 112–125.
- [26] M. Chiarandini, I. Dumitrescu, and T. Stützle, "Stochastic local search algorithms for the graph colouring problem," *Handbook of approximation algorithms and metaheuristics*, pp. 63–1, 2007.
- [27] A. Hertz, M. Plumettaz, and N. Zufferey, "Variable space search for graph coloring," *Discrete Applied Mathematics*, vol. 156, no. 13, pp. 2551–2560, 2008.
- [28] C. Avanthay, A. Hertz, and N. Zufferey, "A variable neighborhood search for graph coloring," *European Journal of Operational Research*, vol. 151, no. 2, pp. 379–388, 2003.
- [29] R. Dorne and J.-K. Hao, "A new genetic local search algorithm for graph coloring," in *International Conference on Parallel Problem Solving from Nature*. Springer, 1998, pp. 745–754.
- [30] P. Galinier and J.-K. Hao, "Hybrid evolutionary algorithms for graph coloring," *Journal of combinatorial optimization*, vol. 3, no. 4, pp. 379–397, 1999.
- [31] K. Han and C. Kim, "An evolutionary approach for graph multi-coloring problem," *Applied Mathematical Sciences*, vol. 9, no. 34, pp. 1677–1684, 2015.
- [32] M. Hindi and R. V. Yampolskiy, "Genetic algorithm applied to the graph coloring problem," in *MAICS*, 2012, pp. 60–66.
- [33] G. Sungu and B. Boz, "An evolutionary algorithm for weighted graph coloring problem," in *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. ACM, 2015, pp. 1233–1236.
- [34] J. W. Shen, "Solving the graph coloring problem using genetic programming," *Genetic Algorithms and Genetic Programming at Stanford*, pp. 187–196, 2003.
- [35] C. A. Glass and A. Prügel-Bennett, "Genetic algorithm for graph coloring: exploration of galinier and hao's algorithm," *Journal of Combinatorial Optimization*, vol. 7, no. 3, pp. 229–236, 2003.
- [36] D. C. Porumbel, J.-K. Hao, and P. Kuntz, "A search space "cartography" for guiding graph coloring heuristics," *Computers & Operations Research*, vol. 37, no. 4, pp. 769–778, 2010.
- [37] C. Oliveira, T. F. Noronha, and S. Urrutia, "Heurística vnd com backtracking para o problema de coloração de vértices com pesos," *Proceedings of the XLIII Simpósio Brasileiro de Pesquisa Operacional*, 2011.
- [38] M. Caramia and P. Dell'Olmo, "Solving the minimum-weighted coloring problem," *Networks*, vol. 38, no. 2, pp. 88–101, 2001.
- [39] J. Cong, Y. Fan, G. Han, Y. Lin, J. Xu, Z. Zhang, and X. Cheng, "Bitwidth-aware scheduling and binding in high-level synthesis," in *Proceedings of the 2005 Asia and South Pacific Design Automation Conference*. ACM, 2005, pp. 856–861.
- [40] J. L. Henning, "Spec cpu2006 benchmark descriptions," *ACM SIGARCH Computer Architecture News*, vol. 34, no. 4, pp. 1–17, 2006.